

Software Design and Architecture

BY PROF. VISHAL NAYAKWADI

Contents

- Design Process, Software design principles, Design Concepts, The Design Model, Software Architecture, architectural Genres, User interface design principles,
- Case Study: Library Management System.

□ Design?

□ What is it?

□ Who does it?

□ Why is it important?

□ What is design in software engineering?

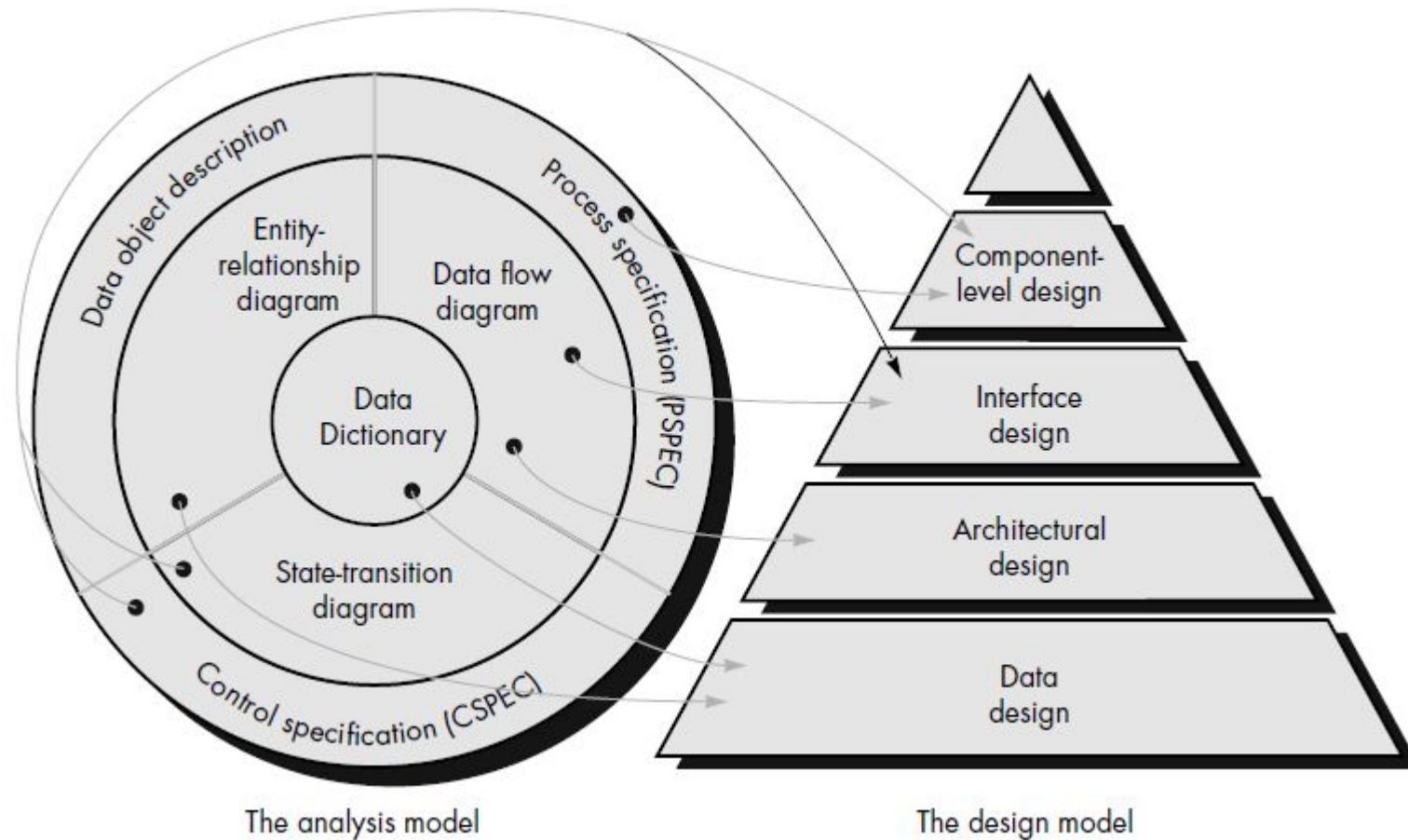
Answers

- “A PLAN OR DRAWING”.
- Software Design -- An iterative process transforming requirements into a “blueprint” for constructing the software
- Software Engineer
- Design allows you to model the system or product that is to be built.
- Software design is the process of implementing software solutions to one or more sets of problems

Design

- “Software Design is model of software which translates the requirement into finished software product in which the details about the software data structures, architecture, interfaces and components that are necessary to implement the system are given.”
- Software design is a process to transform user requirements into some suitable form, which helps the programmer in software coding and implementation.
- Software design is the first step in SDLC (Software Design Life Cycle), which moves the concentration from problem domain to solution domain. It tries to specify how to fulfill the requirements mentioned in SRS

Design Engineering



-
- The data design transforms the information domain model created during analysis into the data structures that will be required to implement the software.
 - The architectural design defines the relationship between major structural elements of the software, the “design patterns” that can be used to achieve the requirements that have been defined for the system.
 - The interface design describes how the software communicates within itself, with systems that interoperate with it, and with humans who use it.
 - The component-level design transforms structural elements of the software architecture into a procedural description of software components.

Design Process

- Software design is an iterative process through which requirements are translated into a “blueprint” for constructing the software
- During the design process the software requirements model is transformed into design models that describe the details of the data structures, system architecture, interface, and components.
- Design is considered to be high level of abstraction.

Design Quality Guidelines

- A design should exhibit an architectural structure that
 - (1) has been created using recognizable design patterns,
 - (2) is composed of components that exhibit good design characteristics (these are discussed later in this chapter), and
 - (3) can be implemented in an evolutionary fashion, thereby facilitating implementation and testing.
- A design should be modular; that is, the software should be logically partitioned into elements that perform specific functions and subfunctions.
- A design should contain distinct representations of data, architecture, interfaces, and components (modules).
- A design should lead to data structures that are appropriate for the objects to be implemented and are drawn from recognizable data patterns.

-
- A design should lead to components that exhibit independent functional characteristics.
 - A design should lead to interfaces that reduce the complexity of connections between modules and with the external environment.
 - A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.

DESIGN PRINCIPLES

- Should not suffer from “Tunnel Vision” –
 - While designing the process, it should not suffer from “tunnel vision” which means that it should not only focus on completing or achieving the aim but on other effects also.
- Traceable to analysis model –
 - The design process should be traceable to the analysis model which means it should satisfy all the requirements that software requires to develop a high-quality product.
- Should not “Reinvent The Wheel” –
 - The design process should not reinvent the wheel that means it should not waste time or effort in creating things that already exist. Due to this, the overall development will get increased.
- Minimize Intellectual distance –
 - The design process should reduce the gap between real-world problems and software solutions for that problem meaning it should simply minimize intellectual distance.
- Exhibit uniformity and integration –
 - The design should display uniformity which means it should be uniform throughout the process without any change. Integration means it should mix or combine all parts of software i.e. subsystems into one system.

□ Accommodate change –

- The software should be designed in such a way that it accommodates the change implying that the software should adjust to the change that is required to be done as per the user's need.

□ Degrade gently –

- The software should be designed in such a way that it degrades gracefully which means it should work properly even if an error occurs during the execution.

□ Assessed or quality –

- The design should be assessed or evaluated for the quality meaning that during the evaluation, the quality of the design needs to be checked and focused on.

□ Review to discover errors –

- The design should be reviewed which means that the overall evaluation should be done to check if there is any error present or if it can be minimized.

□ Design is not coding and coding is not design –

- Design means describing the logic of the program to solve any problem and coding is a type of language that is used for the implementation of a design.

Design Concepts

- Abstraction
- Architecture
- Patterns
- Separation of Concerns
- Modularity
- Information Hiding
- Functional Independence
- Refinement
- Aspects
- Refactoring

Abstraction

- Permits one to concentrate on a problem at some level of abstraction without regard to low level detail.
- Abstraction allows designers to focus on solving a problem without being concerned about irrelevant lower level details.
- There are two types of abstraction available :
 - Procedural abstraction – a sequence of instructions that have a specific and limited function. Eg: Word OPEN for a door.
 - Data abstraction – a named collection of data that describes a data object. Data abstraction for door would be a set of attributes that describes the door (e.g. door type, swing direction, weight, dimension).
 - Control abstraction – a program control mechanism without specifying internal detail. It is used to coordinate all activities in operating system

Architecture

- Overall structure of the system
- The ways in which the structure provides conceptual integrity for a system :
- Consists of components, connectors, and the relationship between them.
- Some of the Architecture models are described below,
 - Structural models – architecture as organized collection of components
 - Framework models – attempt to identify repeatable architectural patterns
 - Dynamic models – indicate how program structure changes as a function of external events
 - Process models – focus on the design of the business or technical process that system must accommodate
 - Functional models – used to represent system functional hierarchy

Patterns

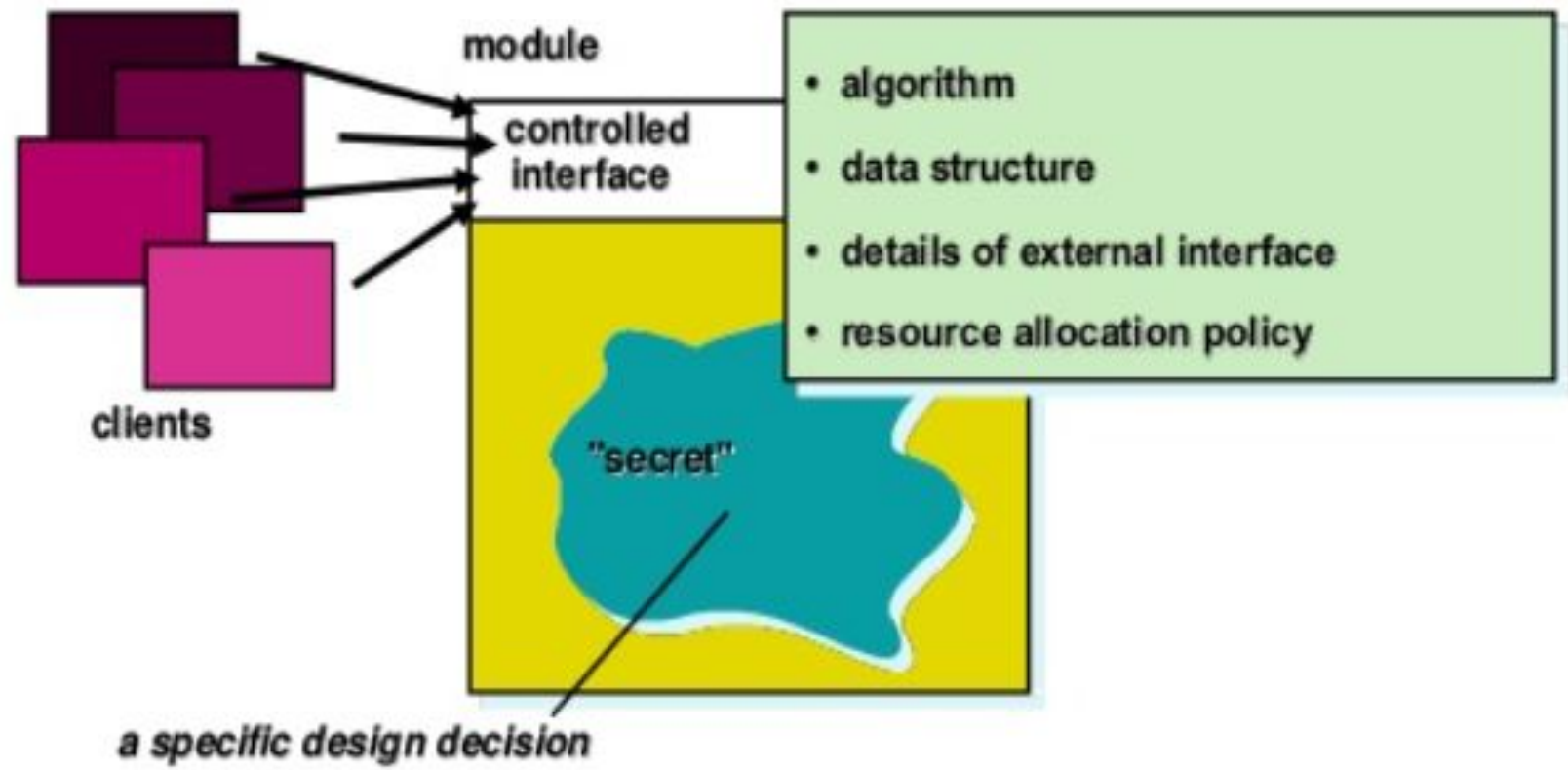
- General reusable solution which can be used again and again
- Software engineer can use the design pattern during the entire software design process.
- Each pattern is to provide an insight to a designer who can determine the following-:
 - Whether the pattern can be reused.
 - Whether the pattern is applicable to the current project.
 - Whether the pattern can be used to develop a similar but functionally or structurally different design pattern.
- Types of Design Patterns
 - Architectural patterns: These patterns are high-level strategies that refer to the overall structure and organization of a software system. That is, they define the elements of a software system such as subsystems, components, classes, etc.
 - Design patterns: These patterns are medium-level strategies that are used to solve design problems. They provide a means for the refinement of the elements of a software system or the relationship among components or mechanisms that affect component-to-component.
 - Idioms: These patterns are low-level patterns, which are programming-language specific. They describe the implementation of a software component, the method used for interaction among software components

Modularity

- Single attribute of software that allows a program to be intellectually manageable. What is the "right" number of modules for a specific software design...???
- Software is divided into separately named and addressable components, often called modules, that are integrated to satisfy problem requirements.
- This leads to a "divide and conquer" conclusion—it's easier to solve a complex problem when you break it into manageable pieces.
- It has been stated that "modularity is the single attribute of software that allows a program to be intellectually manageable“.

Information Hiding

- The way of hiding the unnecessary details
- Information (data and procedure) contained within a module should be inaccessible to other modules that have no need for such information.
- Hiding defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module
- Modules should be specified and designed so that information (procedure and data) contained within a module is inaccessible to other modules that have no need for such information.
- Hiding implies that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve software function.
- This enforces access constraints to both procedural (i.e., implementation) detail and local data structures.



Functional Independence

- Functional Independence Achieved by developing modules with “single minded” function and low coupling
- Functional Independence Functional independence is achieved by developing a module to perform given set of functions without interacting with other parts of the system.
- It can be measured using two criteria:
 - Cohesion
 - A natural extension of the information hiding concept a module may perform a number of tasks.
 - Measures how closely related responsibilities within a single module are (aim for high cohesion)
 - Coupling
 - A measure of interconnection/interdependence among modules in a program structure. Coupling depends on the interface complexity between modules
- Design modules based on independent functional features

Refinement

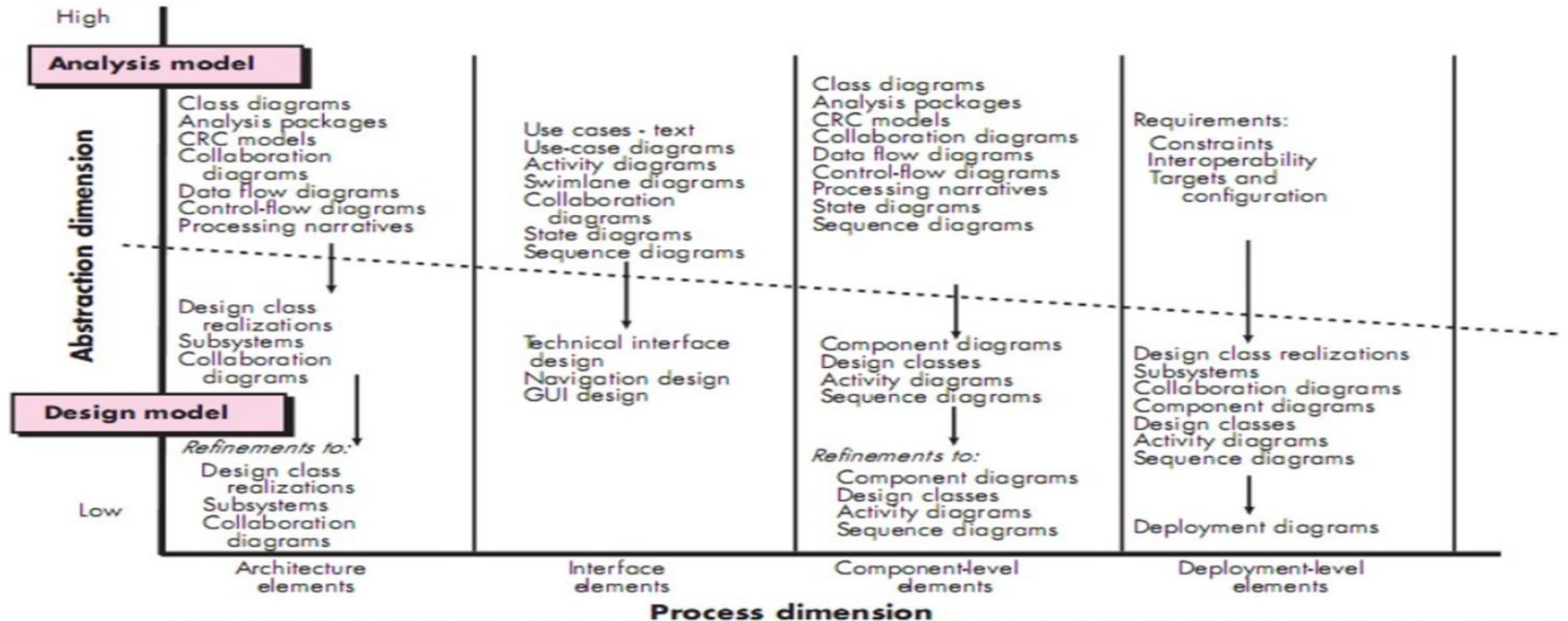
- Stepwise refinement is a top-down design strategy
- Refinement is actually a process of elaboration.
- It begins with a statement of function (or description of information) that is defined at a high level of abstraction and reaches the lower level of abstraction.
- Terminates when all instructions are expressed in terms of any underlying computer or programming language.
- Abstraction and refinement are complementary concepts

Refactoring

- restructuring (rearranging) code... “changing the structure of a program so that the functionality is not changed”
- Refactoring is a reorganization technique that simplifies the design (or internal code structure) of a component without changing its function or external behaviour.
- It removes redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failures

Design Model

Dimensions of the design model



Data design elements

- Data design element produced a model of data that represent a high level of abstraction.
- This model is then more refined into more implementation specific representation which is processed by the computer based system.
- The structure of data is the most important part of the software design.

Architectural design elements

- The architecture design elements provides us overall view of the system.
- The architectural design element is generally represented as a set of interconnected subsystem that are derived from analysis packages in the requirement model.
- The architecture model is derived from following sources:
 - The information about the application domain to built the software.
 - Requirement model elements like data flow diagram or analysis classes, relationship and collaboration between them.
 - The architectural style and pattern as per availability.

Interface design elements

- Interface design elements for software represents the information flow within it and out of the system.
- They communicate between the components defined as part of architecture.
- Following are the important elements of the interface design:
 - The user interface
 - The external interface to the other systems, networks etc.
 - The internal interface between various components.

Component level diagram elements

- The component level design for software is similar to the set of detailed specification of each room in a house.
- The component level design for the software completely describes the internal details of the each software component.
- The processing of data structure occurs in a component and an interface which allows all the component operations.
- In a context of object-oriented software engineering, a component shown in a UML diagram.
- The UML diagram is used to represent the processing logic.



Fig. - UML component diagram for sensor managemnet

Deployment level design elements

- The deployment level design element shows the software functionality and subsystem that allocated in the physical computing environment which support the software.
- Following figure shows three computing environment as shown. These are the personal computer, the CPI server and the Control panel.

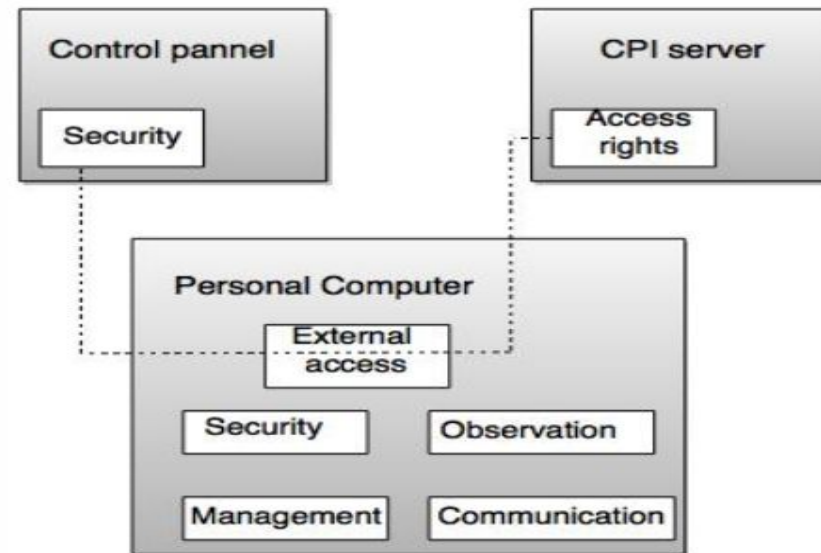
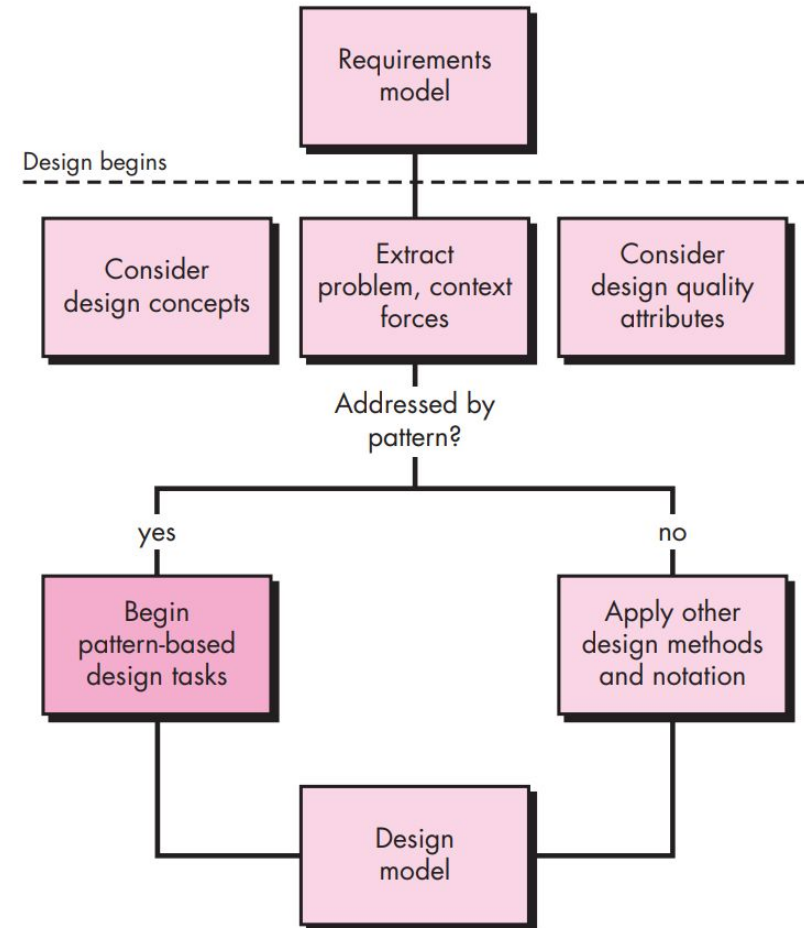


Fig. - Deployment level diagram

Pattern-based Software Design

- Pattern-based design creates a new application by finding a set of proven solutions to a clearly delineated set of problems.
- Each problem and its solution is described by a design pattern that has been cataloged and vetted by other software engineers who have encountered the problem and implemented the solution while designing other applications.
- Each design pattern provides you with a proven approach to one part of the problem to be solved.
- A design model that depicts the architectural structure, user interface, and component-level detail is developed.
- As each design pattern is translated into some element of the design model, work products are reviewed for clarity, correctness, completeness, and consistency with requirements and with one another.



Kinds of Patterns

- Architectural patterns describe broad-based design problems that are solved using a structural approach.
- Data patterns describe recurring data-oriented problems and the data modeling solutions that can be used to solve them.
- Component patterns (also referred to as design patterns) address problems associated with the development of subsystems and components, the manner in which they communicate with one another, and their placement within a larger architecture
- Interface design patterns describe common user interface problems and their solution with a system of forces that includes the specific characteristics of end-users.
- WebApp patterns address a problem set that is encountered when building WebApps and often incorporates many of the other patterns categories just mentioned.

-
- Creational patterns focus on the “creation, composition, and representation of objects, e.g.,
 - Abstract factory pattern: centralize decision of what factory to instantiate
 - Factory method pattern: centralize creation of an object of a specific type choosing one of several implementations
 - Structural patterns focus on problems and solutions associated with how classes and objects are organized and integrated to build a larger structure, e.g.,
 - Adapter pattern: 'adapts' one interface for a class into one that a client expects
 - Aggregate pattern: a version of the Composite pattern with methods for aggregation of children
 - Behavioral patterns address problems associated with the assignment of responsibility between objects and the manner in which communication is effected between objects, e.g.,
 - Chain of responsibility pattern: Command objects are handled or passed on to other objects by logic-containing processing objects
 - Command pattern: Command objects encapsulate an action and its parameters

Architectural Design

- The software architecture of a program or computing system is the structure or structures of the system which comprise
 - The software components
 - The externally visible properties of those components
 - The relationships among the components
- Software architectural design represents the structure of the data and program components that are required to build a computer-based system
- An architectural design model is transferable
 - It can be applied to the design of other systems
 - It represents a set of abstractions that enable software engineers to describe architecture in predictable ways

-
- The architecture is not the operational software. Rather, it is a representation that enables a software engineer to:
 - analyze the effectiveness of the design in meeting its stated requirements
 - consider architectural alternatives at a stage when making design changes is still relatively easy
 - reduce the risks associated with the construction of the software
 - Representations of software architecture are an enabler for communication between all parties (stakeholders) interested in the development of a computer-based system.
 - The architecture highlights early design decisions that will have a profound impact on all software engineering work that follows and, as important, on the ultimate success of the system as an operational entity.
 - Architecture “constitutes a relatively small, intellectually graspable mode of how the system is structured and how its components work together”

Design Decisions

- Architectural design is a creative process where you design a system organization that will satisfy the functional and non-functional requirements of a system.
- Because it is a creative process, the activities within the process depend on the type of system being developed, the background and experience of the system architect, and the specific requirements for the system.
- It is therefore useful to think of architectural design as a series of decisions to be made rather than a sequence of activities.
- During the architectural design process, system architects have to make a number of structural decisions that profoundly affect the system and its development process.

Fundamental Questions About the System

- Is there a generic application architecture that can act as a template for the system that is being designed?
- How will the system be distributed across a number of cores or processors?
- What architectural patterns or styles might be used?
- What will be the fundamental approach used to structure the system?
- How will the structural components in the system be decomposed into subcomponents?
- What strategy will be used to control the operation of the components in the system?
- What architectural organization is best for delivering the non-functional requirements of the system?
- How will the architectural design be evaluated?
- How should the architecture of the system be documented?

Views

- It is impossible to represent all relevant information about a system's architecture in a single architectural model, as each model only shows one view or perspective of the system.
- It might show how a system is decomposed into modules, how the run-time processes interact, or the different ways in which system components are distributed across a network.
- All of these are useful at different times so, for both design and documentation, you usually need to present multiple views of the software architecture.
- There are different opinions as to what views are required.
- Krutchen (1995), in his well-known 4+1 view model of software architecture, suggests that there should be four fundamental architectural views, which are related using use cases or scenarios.

Krutchen Views

□ A logical view

- Which shows the key abstractions in the system as objects or object classes.
- It should be possible to relate the system requirements to entities in this logical view.

□ A process view

- Which shows how, at run-time, the system is composed of interacting processes.
- This view is useful for making judgments about nonfunctional system characteristics such as performance and availability.

□ A development view

- Which shows how the software is decomposed for development, that is, it shows the breakdown of the software into components that are implemented by a single developer or development team.
- This view is useful for software managers and programmers.

□ A physical view

- Which shows the system hardware and how software components are distributed across the processors in the system.
- This view is useful for systems engineers planning a system deployment.

Patterns/Styles

□ System categories define the architectural style

□ Components

- They perform the function

□ Connectors

- Enable communication

□ Constraints

- Define how system can be integrated

□ Semantic models

- Specify how to determine a systems overall properties from the properties of its part

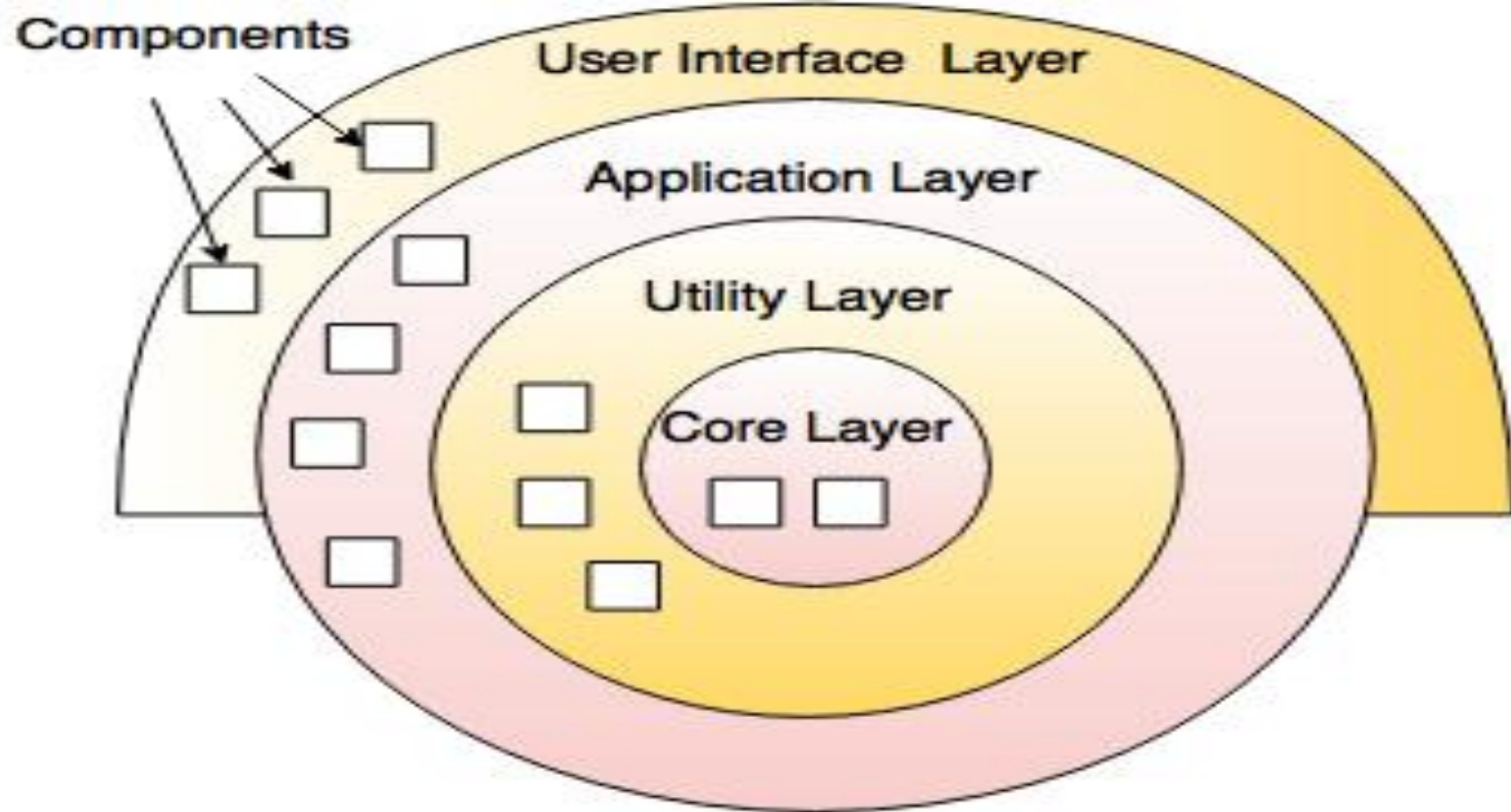
Types of Pattern or Styles (Architectural Genres)

- Layered architecture
- Repository architecture
- Client–server architecture
- Pipe and filter architecture

- Data centered Architectures
- Data flow Architectures
- Call and return architecture
- Object oriented architecture

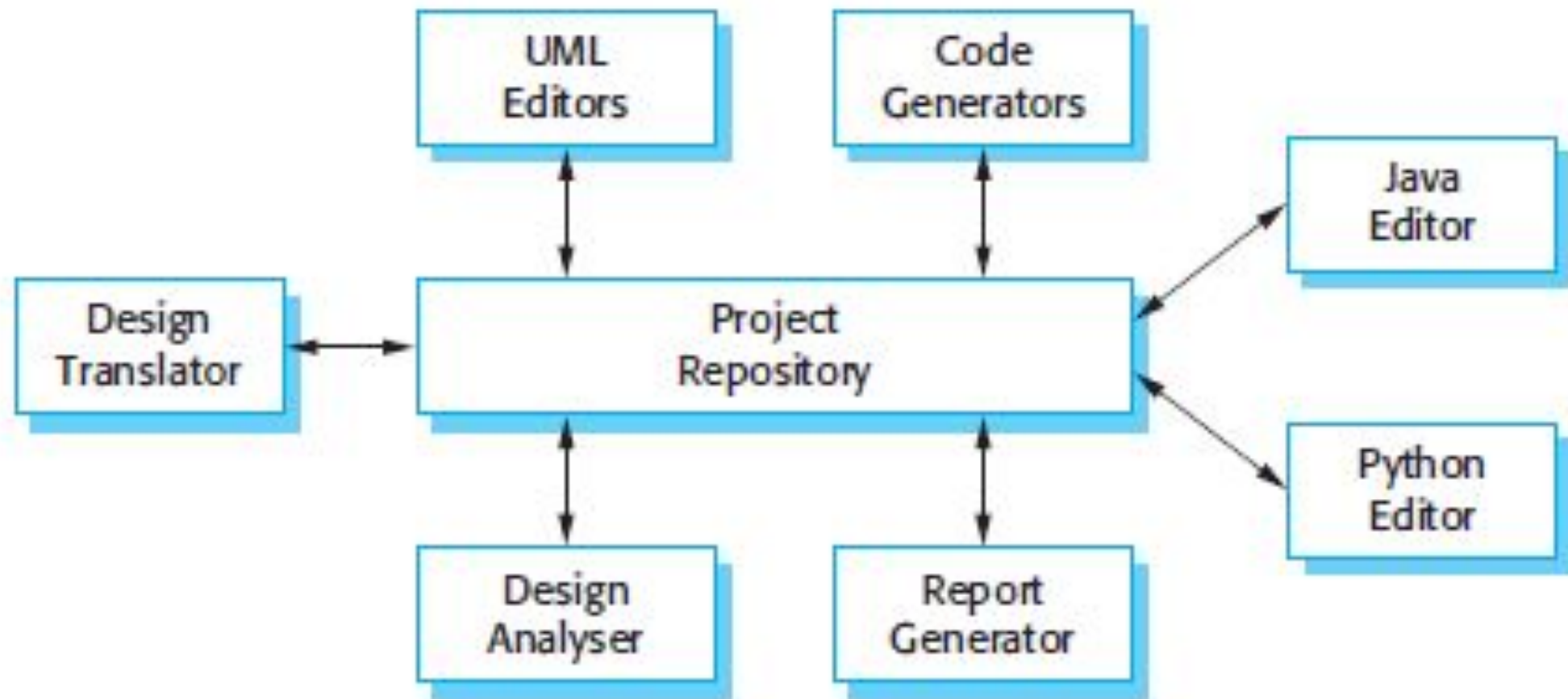
Layered architecture

Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system. See Figure 6.6.
Example	A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.7.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.



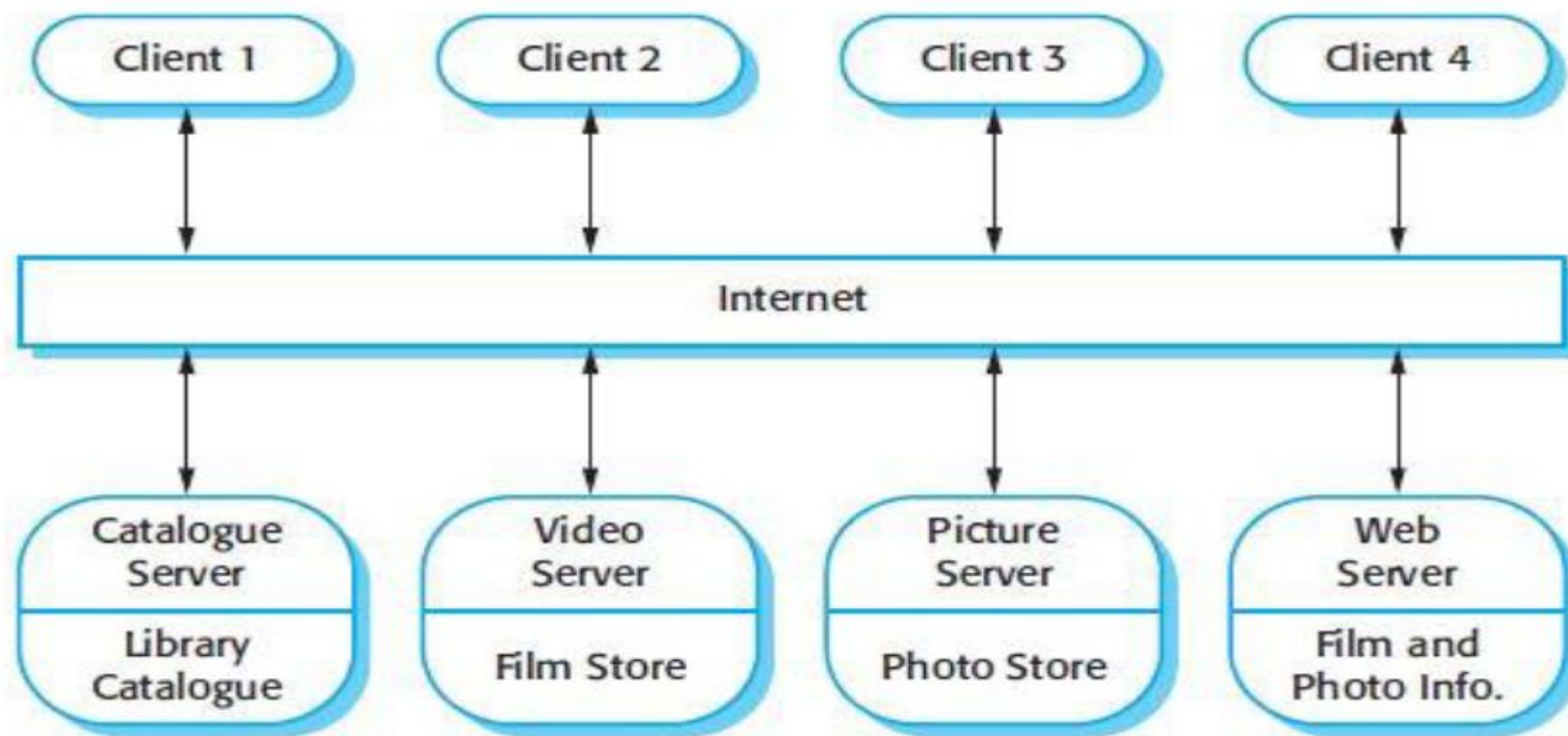
Repository architecture

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.9 is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.



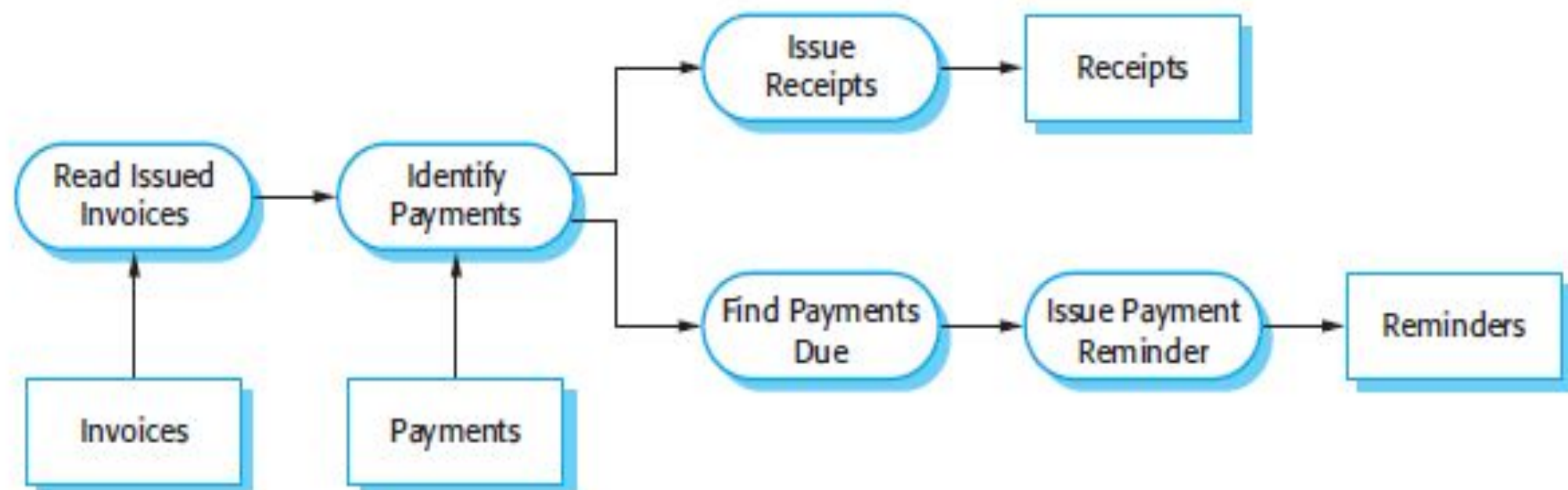
Client-server architecture

Name	Client-server
Description	In a client-server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.11 is an example of a film and video/DVD library organized as a client-server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

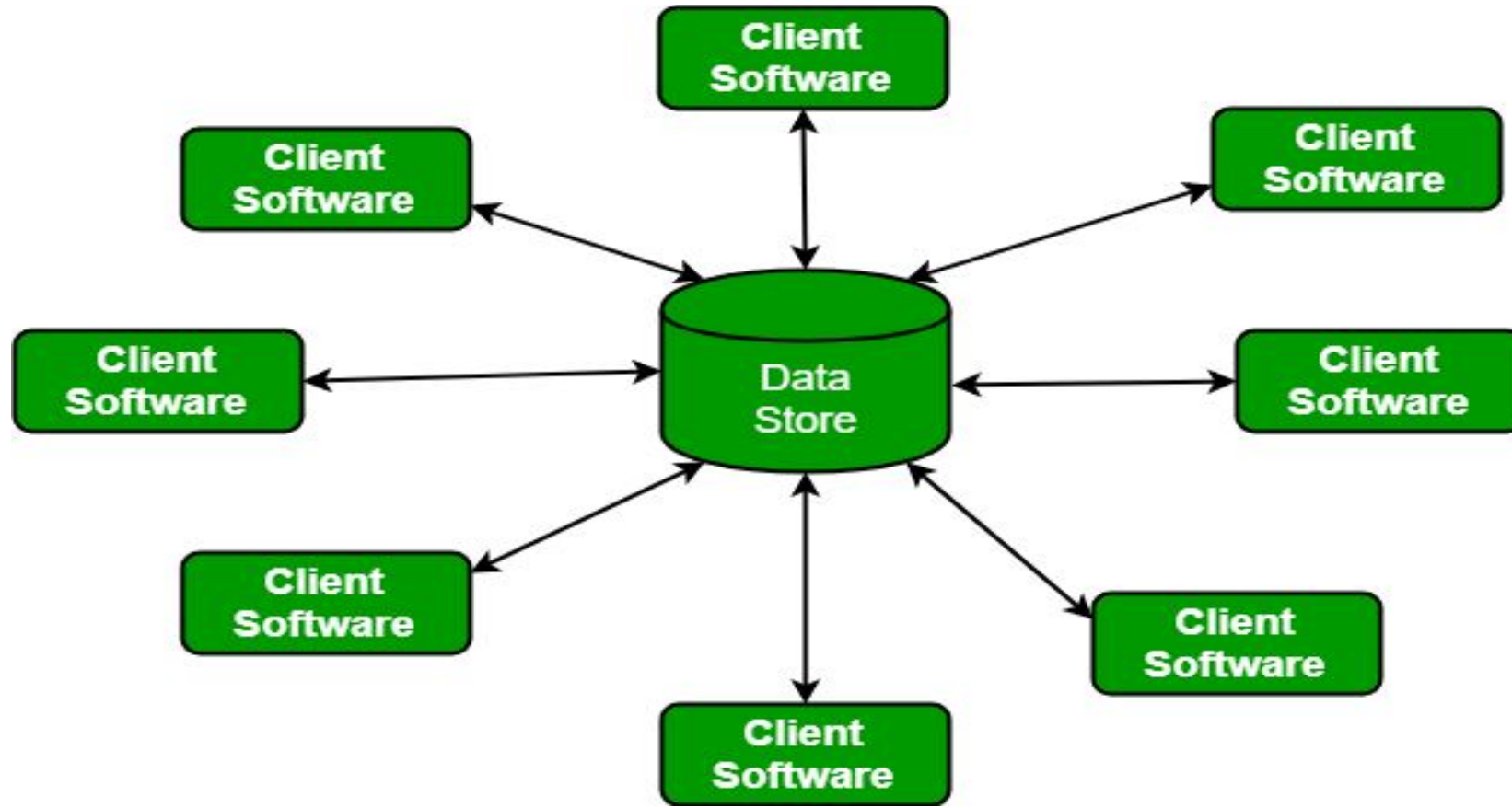


Pipe and filter architecture

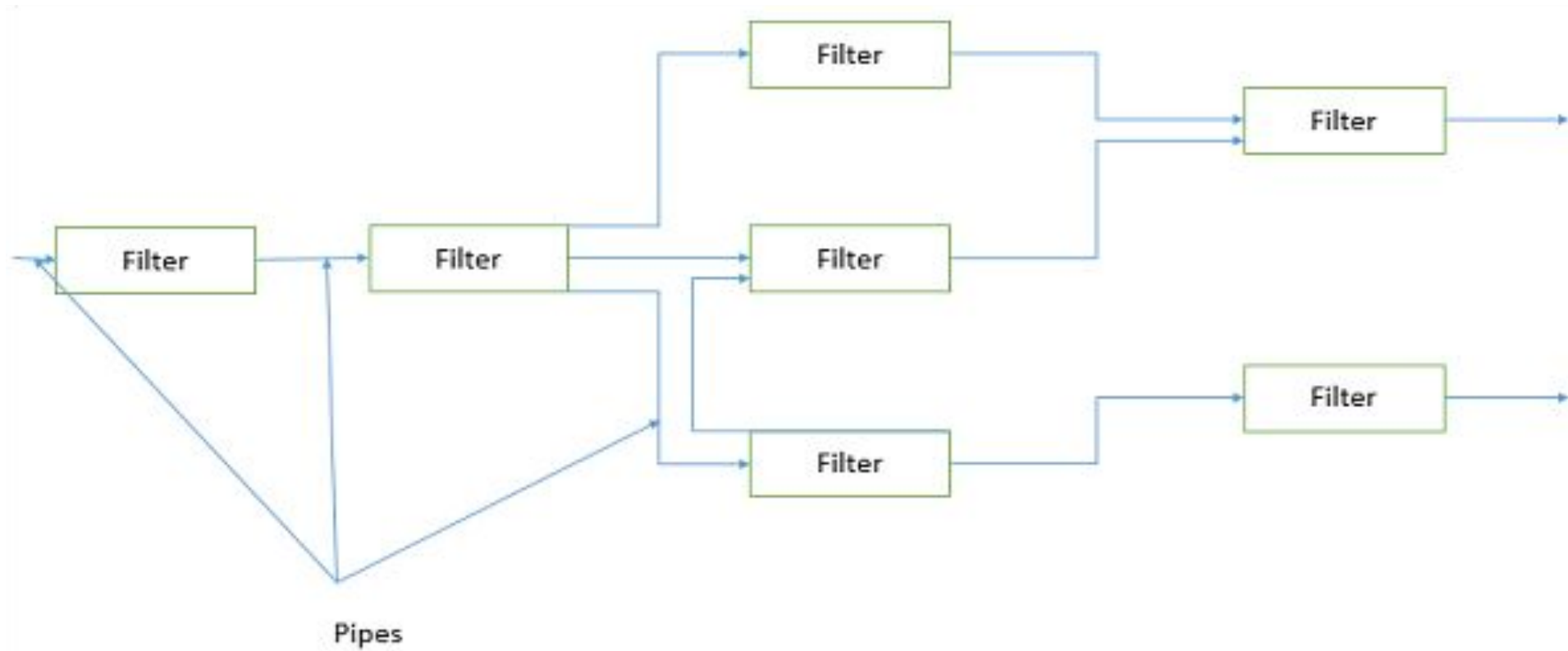
Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.13 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.



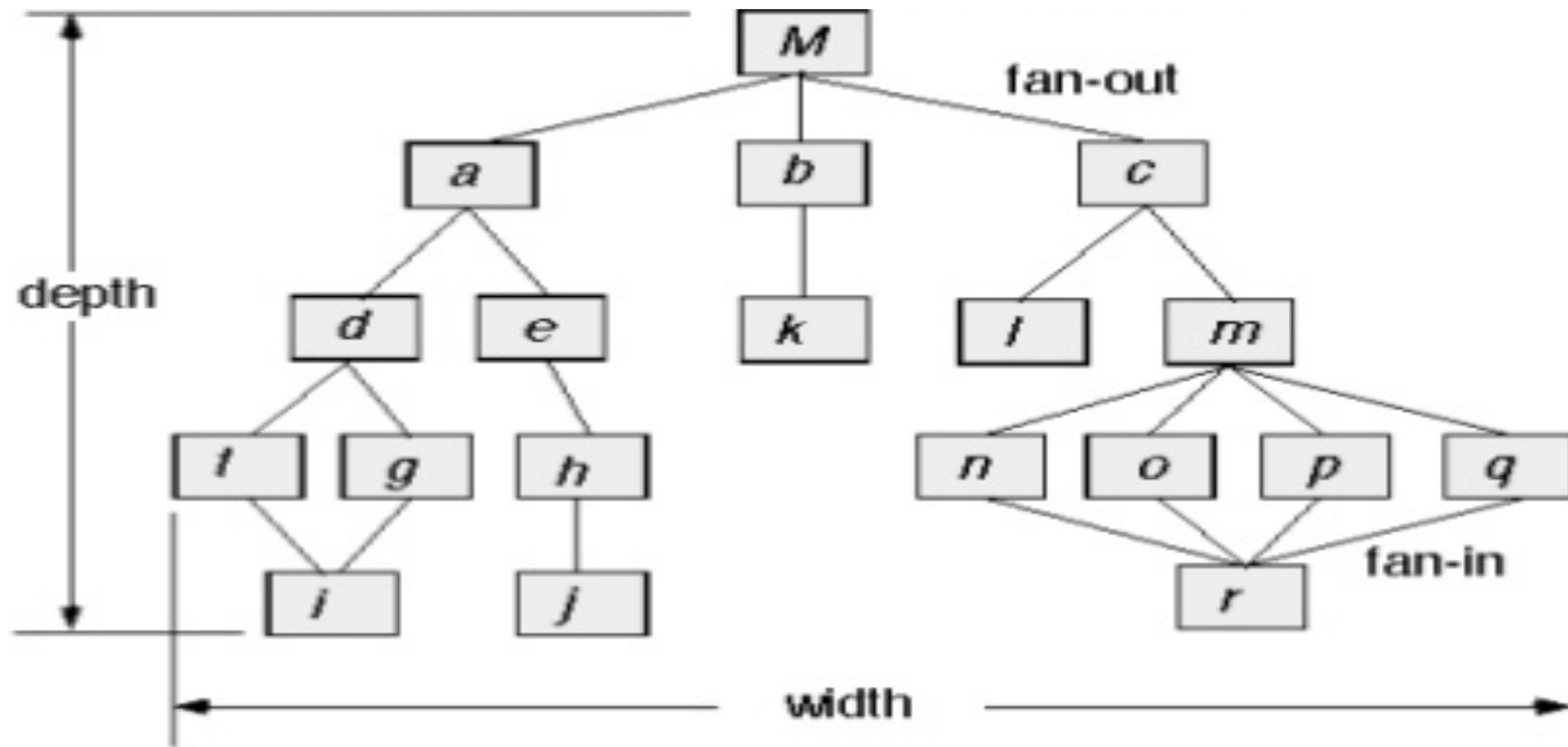
Data centered Architectures



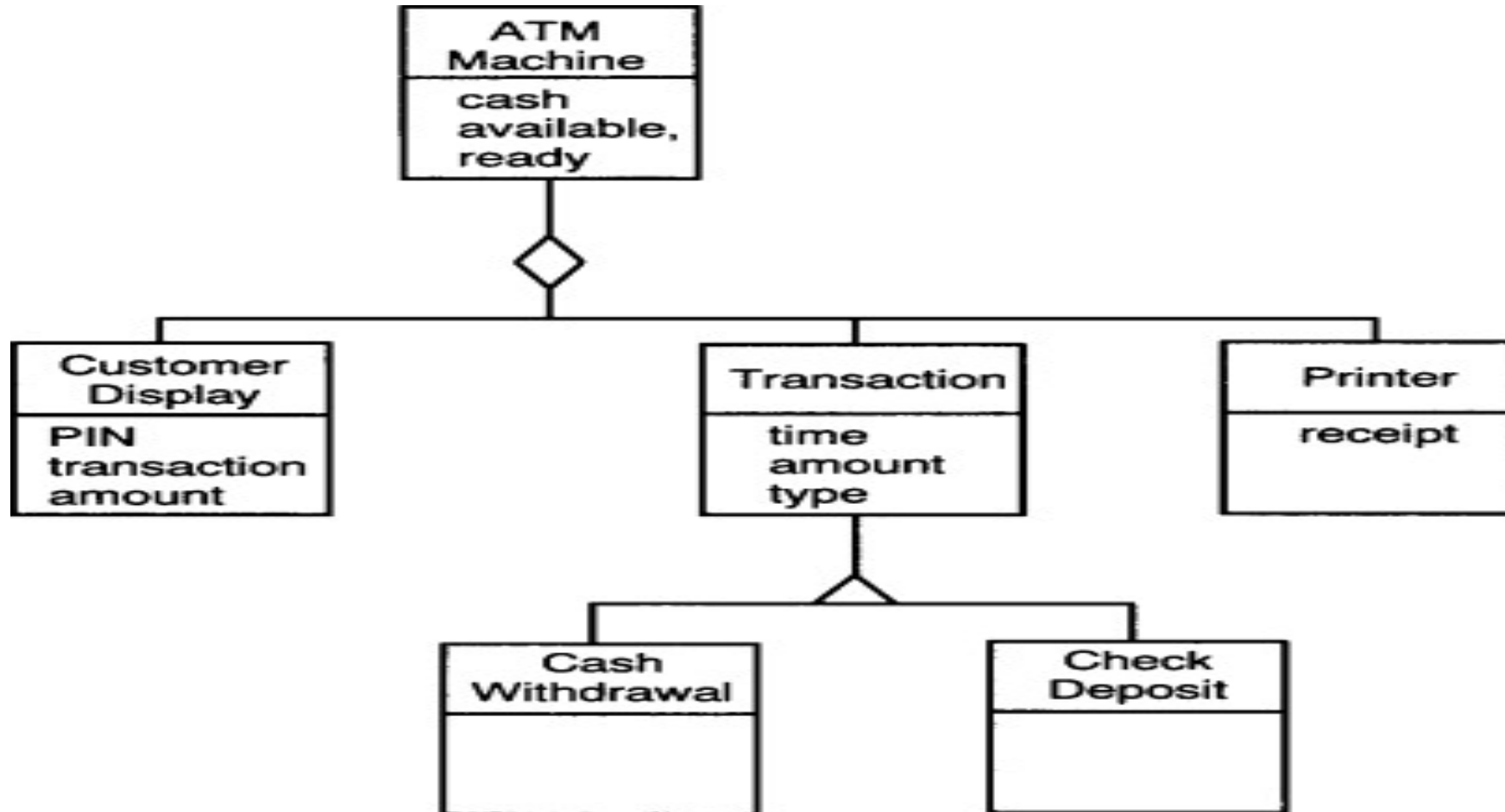
Data flow Architectures



Call and return architecture



Object oriented architecture



Application Architectures

- As a software designer, you can use models of application architectures in a number of ways:
 - As a starting point for the architectural design process If you are unfamiliar with the type of application that you are developing, you can base your initial design on a generic application architecture. Of course, this will have to be specialized for the specific system being developed, but it is a good starting point for design.
 - As a design checklist If you have developed an architectural design for an application system, you can compare this with the generic application architecture. You can check that your design is consistent with the generic architecture.
 - As a way of organizing the work of the development team The application architectures identify stable structural features of the system architectures and in many cases, it is possible to develop these in parallel. You can assign work to group members to implement different components within the architecture.
 - As a means of assessing components for reuse If you have components you might be able to reuse, you can compare these with the generic structures to see whether there are comparable components in the application architecture.
 - As a vocabulary for talking about types of applications If you are discussing a specific application or trying to compare applications of the same types, then you can use the concepts identified in the generic architecture to talk about the applications.

Types

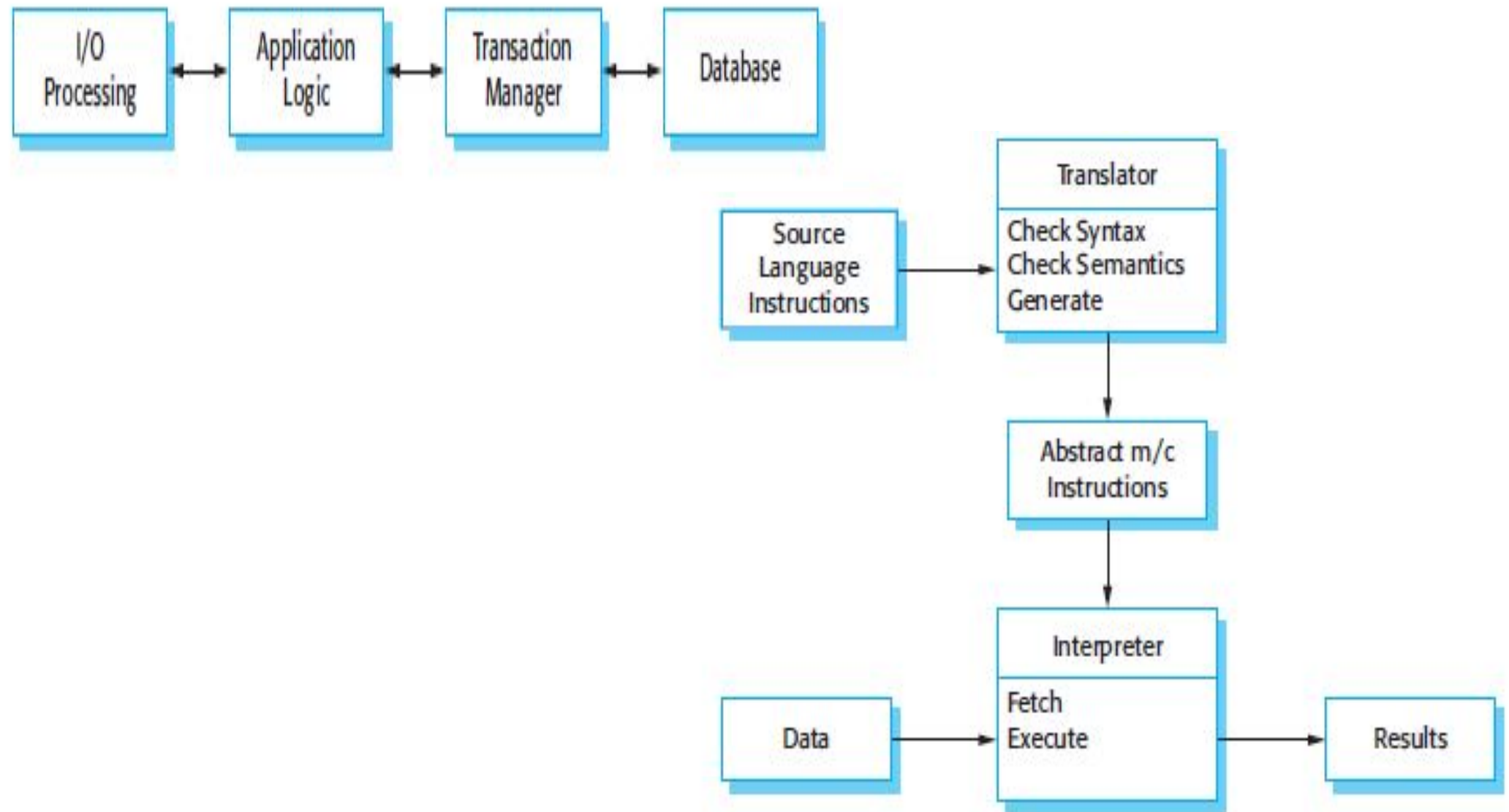
□ Transaction processing applications

- Transaction processing applications are database-centered applications that process user requests for information and update the information in a database.
- These are the most common type of interactive business systems.
- They are organized in such a way that user actions can't interfere with each other and the integrity of the database is maintained.
- This class of system includes interactive banking systems, e-commerce systems, information systems, and booking systems.

□ Language processing systems

- Language processing systems are systems in which the user's intentions are expressed in a formal language (such as Java).
- The language processing system processes this language into an internal format and then interprets this internal representation.
- The best-known language processing systems are compilers, which translate high-level language programs into machine code.

Figure 6.14 The structure of transaction processing applications



Component level Design

- Component design is all about taking complex software systems and making them into small, reusable pieces or simply modules.
- These parts are responsible for directing certain functionalities, so programming them is like building a puzzle with small pieces, which eventually create more complex architectures.
- Component design exhibits a modularized approach to software development where the units are organized systematically to facilitate control of complexity and increase manageability.

Component

□ Szyperski (2002),

- “A software component is a unit of composition with contractually-specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties.”

□ Councill and Heineman (2001) define a component as:

- “A software element that conforms to a standard component model and can be independently deployed and composed without modification according to a composition standard.”

-
- Component—a nontrivial, nearly independent, and replaceable part of a system that fulfills a clear function in the context of a well-defined architecture.
 - Run-time software component—a dynamic bindable package of one or more programs managed as a unit and accessed through documented interfaces that can be discovered in run time.
 - Software component—a unit of composition with contractually specified and explicit context dependencies only.
 - Business component—the software implementation of an “autonomous” business concept or business process.

Component Characteristic	Description
Standardized	Component standardization means that a component used in a CBSE process has to conform to a standard component model. This model may define component interfaces, component metadata, documentation, composition, and deployment.
Independent	A component should be independent—it should be possible to compose and deploy it without having to use other specific components. In situations where the component needs externally provided services, these should be explicitly set out in a 'requires' interface specification.
Composable	For a component to be composable, all external interactions must take place through publicly defined interfaces. In addition, it must provide external access to information about itself, such as its methods and attributes.
Deployable	To be deployable, a component has to be self-contained. It must be able to operate as a stand-alone entity on a component platform that provides an implementation of the component model. This usually means that the component is binary and does not have to be compiled before it is deployed. If a component is implemented as a service, it does not have to be deployed by a user of a component. Rather, it is deployed by the service provider.
Documented	Components have to be fully documented so that potential users can decide whether or not the components meet their needs. The syntax and, ideally, the semantics of all component interfaces should be specified.

Characteristics of Component-Based Design

□ Modularity:

- Some parts of a program retain a certain function or service, essentially serve as reusable software modules, interchangeable and therefore independent components, enhancing flexibility in development, testing, and maintenance.

□ Reusability:

- Component architectural features are envisioned to be adaptable to and reusable on different projects giving rise to shorter development time and lower overheads and making the work of developers much more easier and scientific.

□ Interoperability:

- Components are connected through clearly defined interfaces, ensuring that software systems maintain interaction continuity and carry out various functions within their ecosystems.

□ Encapsulation:

- Components have got encapsulated constructs, which precisely reveal internal details at the interface level, through a provision of only a few interfaces essential for interacting with other components, thus enforcing abstraction by hiding underlying implementation details.

□ Scalability:

- Component-based architectures simplify scalability by providing a way for systems to grow organically via the addition or modification of components without the impacting overall the architecture.

Designing class based components

- Basic design principles are applicable to component-level design and have been widely adopted when object-oriented software engineering is applied.
- The Open-Closed Principle (OCP).
 - “A module [component] should be open for extension but closed for modification” This statement seems to be a contradiction, but it represents one of the most important characteristics of a good component-level design.
- The Liskov Substitution Principle (LSP).
 - “Subclasses should be substitutable for their base classes”. This design principle, originally proposed by Barbara Liskov, suggests that a component that uses a base class should continue to function properly if a class derived from the base class is passed to the component instead. Example :rectangle -> square
- Dependency Inversion Principle (DIP).
 - “Depend on abstractions. Do not depend on concretions”. The more a component depends on other concrete components, the more difficult it will be to extend.

□ The Interface Segregation Principle (ISP).

- “Many client-specific interfaces are better than one general purpose interface”. ISP suggests that you should create a specialized interface to serve each major category of clients.

□ The Release Reuse Equivalency Principle (REP).

- “The granule of reuse is the granule of release”. When classes or components are designed for reuse, there is an implicit contract that is established between the developer of the reusable entity and the people who will use it.

□ The Common Closure Principle (CCP).

- “Classes that change together belong together.” when classes are packaged as part of a design, they should address the same functional or behavioral area. When some characteristic of that area must change, it is likely that only those classes within the package will require modification.

□ The Common Reuse Principle (CRP).

- “Classes that aren’t reused together should not be grouped together”. When one or more classes within a package changes, the release number of the package changes.

Conducting component-level design

- The following steps represent a typical task set for component-level design, when it is applied for an object-oriented system.
 - Step 1 - Identify all design classes that correspond to the problem domain.
 - Step 2 - Identify all design classes that correspond to the infrastructure domain.
 - Step 3 - Elaborate all design classes that are not acquired as reusable components.
 - Step 3a - Specify message details when classes or components collaborate.
 - Step 3c - Elaborate attributes and define data types and data structures required to implement them. Step 3d - Describe processing flow within each operation in detail.
 - Step 4 - Describe persistent data sources (databases and files) and identify the classes required to manage them.
 - Step 5 - Develop and elaborate behavioral representations for a class or component
 - Step 6 -Elaborate deployment diagrams to provide additional implementation detail
 - Step 7 - Refactor every component-level design representation and always consider alternatives.

User Interface Design

□ What is it?

- User interface design creates an effective communication medium between a human and a computer.
- Following a set of interface design principles, design identifies interface objects and actions and then creates a screen layout that forms the basis for a user interface prototype.

□ Who does it?

- A software engineer designs the user interface by applying an iterative process that draws on predefined design principles.

□ Why is it important?

- If software is difficult to use, if it forces you into mistakes, or if it frustrates your efforts to accomplish your goals, you won't like it, regardless of the computational power it exhibits or the functionality it offers. Because it molds a user's perception of the software, the interface has to be right.

□ What are the steps?

- User interface design begins with the identification of user, task, and environmental requirements. Once user tasks have been identified, user scenarios are created and analyzed to define a set of interface objects and actions.

The golden rules

- 1. Place the user in control.
 - During a requirements-gathering session for a major new information system, a key user was asked about the attributes of the window-oriented graphical interface
- 2. Reduce the user's memory load.
 - The more a user has to remember, the more error-prone will be the interaction with the system.
- 3. Make the interface consistent.

Place the user in control.

- Define interaction modes in a way that does not force a user into unnecessary or undesired actions.
 - An interaction mode is the current state of the interface.
 - For example, if spell check is selected in a word-processor menu, the software moves to a spell checking mode.
- Provide for flexible interaction.
 - Because different users have different interaction preferences, choices should be provided.
 - For example, software might allow a user to interact via keyboard commands, mouse movement, a digitizer pen, or voice recognition commands.
- Allow user interaction to be interruptible and undoable.
 - Even when involved in a sequence of actions, the user should be able to interrupt the sequence to do something else (without losing the work that had been done).
 - The user should also be able to “undo” any action.

□ Streamline interaction as skill levels advance and allow the interaction to be customized.

□ Users often find that they perform the same sequence of interactions repeatedly.

□ It is worthwhile to design a “macro” mechanism that enables an advanced user to customize the interface to facilitate interaction.

□ Hide technical internals from the casual user.

□ The user interface should move the user into the virtual world of the application.

□ The user should not be aware of the operating system, file management functions, or other arcane computing technology.

□ Design for direct interaction with objects that appear on the screen.

□ The user feels a sense of control when able to manipulate the objects that are necessary to perform a task in a manner similar to what would occur if the object were a physical thing.

□ For example, an application interface that allows a user to “stretch” an object (scale it in size) is an implementation of direct manipulation.

Reduce the user's memory load.

- Reduce demand on short-term memory.

- When users are involved in complex tasks, the demand on short-term memory can be significant. The interface should be designed to reduce the requirement to remember past actions and results.

- Establish meaningful defaults.

- The initial set of defaults should make sense for the average user, but a user should be able to specify individual preferences. However, a “reset” option should be available, enabling the redefinition of original default values. Define shortcuts that are intuitive. When mnemonics are used to accomplish a system function (e.g., alt-P to invoke the print function), the mnemonic should be tied to the action in a way that is easy to remember (e.g., first letter of the task to be invoked).

- The visual layout of the interface should be based on a real world metaphor.

- For example, a bill payment system should use a check book and check register metaphor to guide the user through the bill paying process. This enables the user to rely on well-understood visual cues, rather than memorizing an arcane interaction sequence.

- Disclose information in a progressive fashion.

- The interface should be organized hierarchically. That is, information about a task, an object, or some behavior should be presented first at a high level of abstraction.

Make the Interface Consistent

- The interface should present and acquire information in a consistent fashion.
- This implies that
 - all visual information is organized according to a design standard that is maintained throughout all screen displays,
 - input mechanisms are constrained to a limited set that are used consistently throughout the application, and
 - mechanisms for navigating from task to task are consistently defined and implemented.

-
- Allow the user to put the current task into a meaningful context.
 - Many interfaces implement complex layers of interactions with dozens of screen images.
 - It is important to provide indicators (e.g., window titles, graphical icons, consistent color coding) that enable the user to know the context of the work at hand.
 - Maintain consistency across a family of applications.
 - A set of applications (or products) should all implement the same design rules so that consistency is maintained for all interaction.
 - If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so.
 - Once a particular interactive sequence has become a de facto standard (e.g., the use of alt-S to save a file), the user expects this in every application he encounters.
 - A change (e.g., using alt-S to invoke scaling) will cause confusion.

User Interface Design Principles

□ User-Centered Design

- Design should focus on users' needs, skills, and expectations.
- Conduct user analysis, personas, and usability testing.
- System should match real-world tasks.

□ Consistency

- Maintain uniform layout, colors, fonts, and terminology.
- Similar actions should produce similar results.
- Reduces learning time and confusion.

□ Simplicity

- Avoid unnecessary elements and complexity.
- Keep interface clean and minimal.
- “Less is more” principle.

☐ Visibility

- ☐ Important functions and information should be clearly visible.
- ☐ Users should not search for key features.
- ☐ Example: Clearly visible navigation menu.

☐ Feedback

- ☐ System must respond to user actions.
- ☐ Show messages for success, error, loading, etc.
- ☐ Example: “File uploaded successfully.”

☐ Error Prevention and Handling

- ☐ Prevent errors through validation.
 - ☐ Provide meaningful error messages.
 - ☐ Allow undo/redo options.
- 

□ Flexibility and Efficiency

- Support both novice and expert users.
- Provide shortcuts, auto-complete, and customization.

□ Learnability

- Interface should be easy to learn.
- Use familiar icons and symbols.
- Provide help/documentation when required.

□ Accessibility

- Design for users with disabilities.
- Use proper contrast, readable fonts, keyboard navigation.
- Follow accessibility standards (e.g., WCAG guidelines).

□ Aesthetic and Minimalist Design

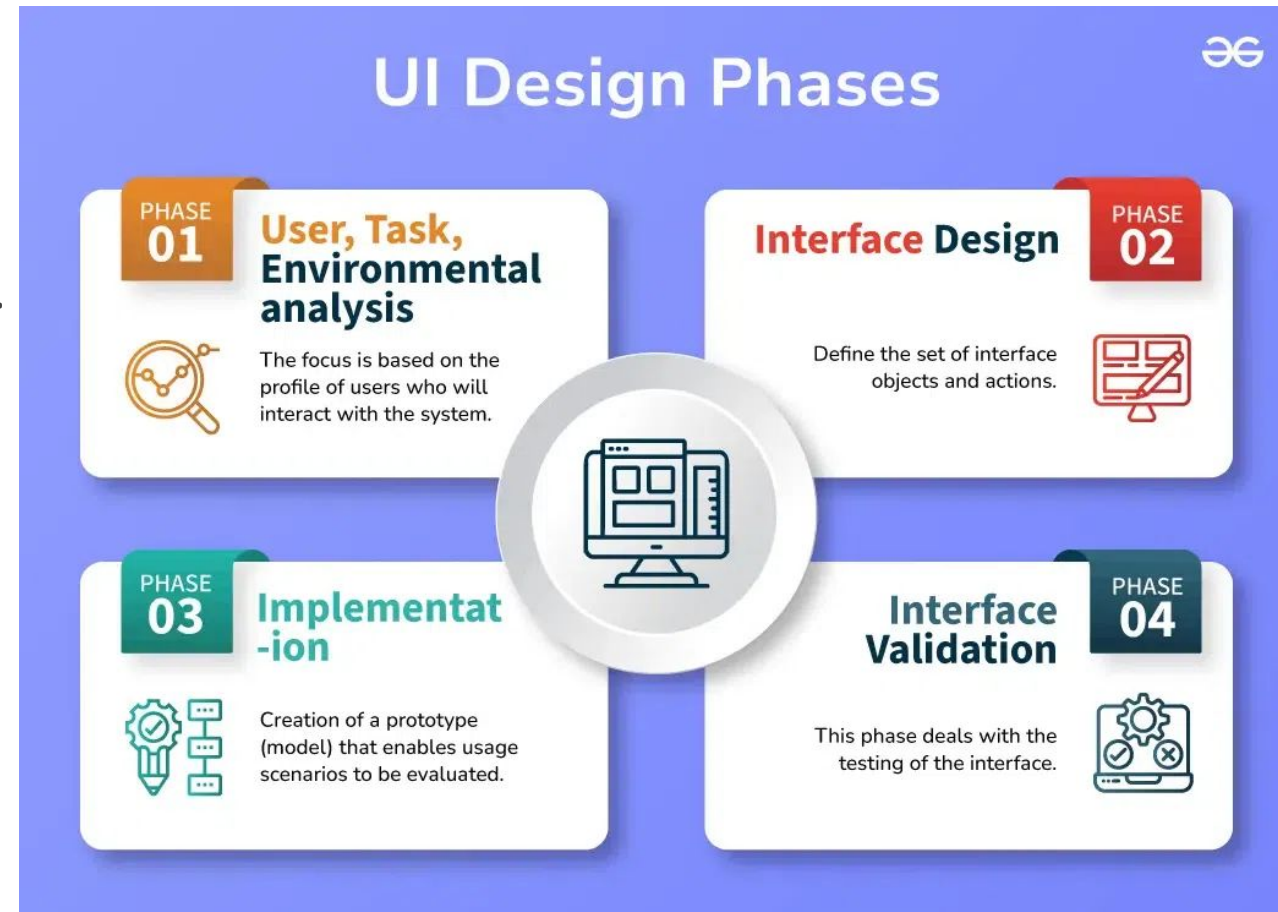
- Avoid clutter.
 - Show only relevant information.
 - Improve readability and user satisfaction.
- 

Shneiderman's Eight Golden Rules for Interface Design

- Strive for consistency
- Enable shortcuts
- Offer informative feedback
- Design dialogs to yield closure
- Prevent errors
- Permit easy reversal of actions
- Support internal locus of control
- Reduce short-term memory load

Interface Design steps & Analysis

- The analysis and design process of a user interface is iterative and can be represented by a spiral model.
- The analysis and design process of user interface consists of four framework activities.



□ User, Task, Environmental Analysis, and Modeling

- Initially, the focus is based on the profile of users who will interact with the system, i.e., understanding, skill and knowledge, type of user, etc., based on the user's profile users are made into categories.
- From each category requirements are gathered.
- Based on the requirement's developer understand how to develop the interface.
- Once all the requirements are gathered a detailed analysis is conducted. In the analysis part, the tasks that the user performs to establish the goals of the system are identified, described and elaborated.
- The analysis of the user environment focuses on the physical work environment.
- Among the questions to be asked are:
 - Where will the interface be located physically?
 - Will the user be sitting, standing, or performing other tasks unrelated to the interface?
 - Does the interface hardware accommodate space, light, or noise constraints?
 - Are there special human factors considerations driven by environmental factors?

□ Interface Design

- The goal of this phase is to define the set of interface objects and actions i.e., control mechanisms that enable the user to perform desired tasks. Indicate how these control mechanisms affect the system.
- Specify the action sequence of tasks and subtasks, also called a user scenario.
- Indicate the state of the system when the user performs a particular task. Always follow the three golden rules stated by Theo Mandel.
- Design issues such as response time, command and action structure, error handling, and help facilities are considered as the design model is refined.
- This phase serves as the foundation for the implementation phase.

□ Interface Construction and Implementation

- The implementation activity begins with the creation of a prototype (model) that enables usage scenarios to be evaluated.
- As iterative design process continues a User Interface toolkit that allows the creation of windows, menus, device interaction, error messages, commands, and many other elements of an interactive environment can be used for completing the construction of an interface.

□ Interface Validation

- This phase focuses on testing the interface.
- The interface should be in such a way that it should be able to perform tasks correctly, and it should be able to handle a variety of tasks.
- It should achieve all the user's requirements.
- It should be easy to use and easy to learn. Users should accept the interface as a useful one in their work.

Design Evaluation

- Once an operational user interface prototype has been created, it must be evaluated to determine whether it meets the needs of the user.
- Evaluation can span a formality spectrum that ranges from an informal "test drive," in which a user provides impromptu feedback to a formally designed study that uses statistical methods for the evaluation of questionnaires completed by a population of end-users.
- After the design model has been completed, a first-level prototype is created.
- The prototype is evaluated by the user, who provides the designer with direct comments about the efficacy of the interface. In addition, if formal evaluation techniques are used (e.g., questionnaires, rating sheets), the designer may extract information from these data.
- Design modifications are made based on user input and the next level prototype is created.
- The evaluation cycle continues until no further modifications to the interface design are necessary.

FIGURE 15.3

The interface design evaluation cycle

